

DGX Spark Inference API — Integration Guide

OpenAI-compatible inference for **17 local models**, reachable from anywhere on the internet. Any app or SDK that speaks the OpenAI API works unchanged — just point `base_url` here and use the bearer token.

1. Connection details

Field	Value
Base URL (public)	<code>https://spark-e257.tail803c7f.ts.net:8443/v1</code>
Base URL (tailnet)	<code>http://spark-e257.tail803c7f.ts.net:8010/v1</code>
API key	bearer token — see <code>models/router-api-key.txt</code> on the box
Auth header	<code>Authorization: Bearer <token></code>
Protocol	OpenAI Chat Completions (<code>/v1/chat/completions</code> , <code>/v1/models</code>)

The public URL is served via **Tailscale Funnel** (real TLS cert, no VPN needed on the client). The tailnet URL is faster (no funnel hop) if the caller is already on the tailnet.

Guard the token. Anyone holding it can use the GPU. There is no rate limit. To rotate: regenerate `models/router-api-key.txt` , update `compose/.env` (`ROUTER_API_KEY`), then restart `vllm-router` + `open-webui` .

2. Models

Pass one of these exact strings as `"model"` :

Model ID	Params	tok/s	Image	Video	Best at
qwen3.6-35b-a3b-nvidia-nvfp4	35B/3B MoE	114	–	–	fastest; bulk generation, agents
kimi-linear-48b-a3b-gguf	48B/3B MoE	73	–	–	fast general chat, long context
nemotron-3.5-lightning-30b-a3b-nvidia-nvfp4	30B/3B MoE	74	–	–	fast reasoning ; 1M context, hybrid Mamba (NVIDIA)
qwen3-vl-30b-a3b-gguf	30B/3B MoE	72			vision at speed — OCR, screenshots
gemma-4-26b-a4b-unsloth-nvfp4	26B/4B MoE	39	.	–	lowest latency (0.09s TTFT)
qwen3.6-35b-a3b-qwen-fp8	35B/3B MoE	28	–	–	best coding (88.4% HumanEval+)
gemma-4-26b-a4b-nvidia-nvfp4	26B/4B MoE	28		–	NVIDIA Gemma-4 quant, low latency
qwen3-next-80b-thinking-cyankiwi-awq	80B/3B MoE	27	–	–	reasoning — step-by-step
qwen3-next-80b-instruct-nvidia-nvfp4	80B/3B MoE	22	–	–	large-model knowledge, general
glm-4.6v-awq-4bit	106B/12B MoE	20	.	.	strongest multimodal
laguna-s-2.1-poolside-nvfp4	118B/8B MoE	18	–	–	agentic coding (SWE-bench 78.5%)
qwen3.6-27b-nvidia-nvfp4	27B dense	11	.	.	dense quality + vision, 256K ctx
qwen3-vl-32b-instruct-gguf	32B dense	10			highest-quality vision
	27B dense	10			

Model ID	Params	tok/s	Image	Video	Best at
qwen3.8-27b-unsloth-nvfp4					newest Qwen dense VLM, 256K ctx
gemma-4-31b-it-nvidia-nvfp4	31B dense	7		–	largest dense Gemma
internvl3-78b-gguf	78B dense	4		–	MIT-licensed VLM, ~72% MMMU
muse-glimmer-30b-gguf	30B dense	13		–	reasoning + vision ; strong chart/diagram reading, 131K ctx

Live version of this table (auto-updating, shows which model is loaded): <https://spark-e257.tail803c7f.ts.net:8443/models> · JSON at </catalog>.

Fetch the list programmatically:

```
curl -s https://spark-e257.tail803c7f.ts.net:8443/v1/models \
-H "Authorization: Bearer $DGX_API_KEY"
```

3. Model swapping (important for API users)

The box has **121 GB unified memory** — **only ONE model is loaded at a time**. There is no separate "load" API: **swapping is automatic and implicit**.

- Request a model that's **already loaded** → answers immediately.
- Request a **different** model → the router stops the current one, starts the requested one, warms it, then serves your request **on the same HTTP call**. Cold swap takes **~1-5 minutes** (weights load + kernel warmup).
- Request an **unknown** model → `404 model_not_found` with the valid list.

Handling cold swaps

Use streaming (`"stream": true`) — the router emits live progress (`Loading ... 45% ... Ready in 210s`) as assistant content, then the real answer. This keeps the connection alive and avoids client timeouts.

```
curl -N https://spark-e257.tail803c7f.ts.net:8443/v1/chat/completions \
-H "Authorization: Bearer $DGX_API_KEY" -H "Content-Type: application/json" \
-d '{"model": "qwen3-vl-30b-a3b-gguf", "stream": true,
  "messages": [{"role": "user", "content": "hi"}]}'
```

Non-streaming works too, but the call simply blocks (no progress output) for the whole load. **Set a client timeout of at least 600 s** or you'll abort a perfectly good cold start.

Explicitly pre-loading a model (no CLI needed)

Send a tiny throwaway request; when it returns, the model is warm:

```
curl -s https://spark-e257.tail803c7f.ts.net:8443/v1/chat/completions \
-H "Authorization: Bearer $DGX_API_KEY" -H "Content-Type: application/json" \
-d '{"model":"glm-4.6v-awq-4bit","messages":[{"role":"user","content":"hi"}],
  "max_tokens":1}' --max-time 900
```

Best practice for applications

- **Pin one model** per application if you can. Mixed traffic across models causes repeated swaps (each 1–5 min) and will feel broken.
- Concurrent requests for **different** models serialize behind swaps. Concurrent requests for the **same** model batch normally and are fast.
- Check what's loaded first (`/health`) and prefer it when the choice is free.

4. Health & status

Endpoint	Auth	Returns
<code>GET /health</code>	none	<code>{"status":"ok","loaded":["<model-id>"]}</code>
<code>GET /catalog</code>	none	full model metadata + which is loaded (JSON)
<code>GET /models</code>	none	human-readable comparison dashboard (HTML)
<code>GET /v1/models</code>	bearer	OpenAI-style model list

```
# is the server up, and which model is hot?
curl -s https://spark-e257.tail803c7f.ts.net:8443/health
# -> {"status":"ok","loaded":["glm-4.6v-awq-4bit"]}
```

`/health` needs no token — safe for uptime monitors and load-balancer probes. An empty `loaded` array means no model is currently resident (idle); the next request will load one.

5. Code examples

curl — chat

```
curl https://spark-e257.tail803c7f.ts.net:8443/v1/chat/completions \
-H "Authorization: Bearer $DGX_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "model": "qwen3.6-35b-a3b-nvidia-nvfp4",
  "messages": [{"role": "user", "content": "Explain vector databases briefly."}],
  "max_tokens": 500,
  "temperature": 0.7
}'
```

Python — OpenAI SDK

```
from openai import OpenAI
import os

client = OpenAI(
    base_url="https://spark-e257.tail803c7f.ts.net:8443/v1",
    api_key=os.environ["DGX_API_KEY"],
    timeout=900,                      # allow for cold model swaps
)

r = client.chat.completions.create(
    model="qwen3.6-35b-a3b-nvidia-nvfp4",
    messages=[{"role": "user", "content": "Explain vector databases briefly."}],
)

print(r.choices[0].message.content)
```

Python — streaming (recommended)

```
stream = client.chat.completions.create(
    model="glm-4.6v-awq-4bit",
    messages=[{"role": "user", "content": "hello"}],
    stream=True,
)

for chunk in stream:
    delta = chunk.choices[0].delta.content or ""
    print(delta, end="", flush=True)  # includes load-progress text on cold swap
```

Vision (image input)

Any model with `vision` in the Image column. Standard OpenAI multimodal format:

```
import base64

img = base64.b64encode(open("photo.jpg", "rb").read()).decode()

r = client.chat.completions.create(
    model="qwen3-vl-30b-a3b-gguf",
    messages=[{"role": "user", "content": [
        {"type": "text", "text": "What's in this image?"},
        {"type": "image_url",
         "image_url": {"url": f"data:image/jpeg;base64,{img}"}}
    ]}],
)
```

Node / TypeScript

```
import OpenAI from "openai";
const client = new OpenAI({
  baseURL: "https://spark-e257.tail803c7f.ts.net:8443/v1",
  apiKey: process.env.DGX_API_KEY,
  timeout: 900_000,
});
const r = await client.chat.completions.create({
  model: "kimi-linear-48b-a3b-gguf",
  messages: [{ role: "user", content: "hi" }],
});
console.log(r.choices[0].message.content);
```

LangChain

```
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(
    base_url="https://spark-e257.tail803c7f.ts.net:8443/v1",
    api_key=os.environ["DGX_API_KEY"],
    model="qwen3.6-35b-a3b-nvidia-nvfp4",
    timeout=900,
)
```

LiteLLM

Use the `openai/` prefix so LiteLLM treats it as a custom OpenAI-compatible host:

```
import litellm
litellm.completion(
    model="openai/qwen3.6-35b-a3b-nvidia-nvfp4",
    api_base="https://spark-e257.tail803c7f.ts.net:8443/v1",
    api_key=os.environ["DGX_API_KEY"],
    messages=[{"role": "user", "content": "hi"}],
    timeout=900,
)
```

A LiteLLM **proxy** also runs on the box at `:4000` (tailnet-only) with all models pre-configured — see `litellm/config.yaml`.

6. Model-specific notes

- **Reasoning models** (`qwen3-next-80b-thinking-*`, `glm-4.6v-*`, `qwen3.6-*`, `muse-glimmer-30b-gguf`): they "think" before answering. Thinking text arrives in `message.reasoning` (vLLM models) or `message.reasoning_content` (llama.cpp models e.g. Muse-Glimmer) — **not** in `content`. Give them generous `max_tokens` (≥ 600) or you may get `finish_reason: "length"` with `content: null`. Disable thinking for fast, direct answers: `json "chat_template_kwargs": {"enable_thinking": false}`
- **GLM-4.6V** wraps its final answer in `<|begin_of_box|> ... <|end_of_box|>` markers — strip them if you need clean text.
- **Tool calling** works on all models (`tools` + `tool_choice`, OpenAI format); each model is served with its correct parser.
- **Context**: most are configured at 32K–64K. The dense `qwen3.6-27b` supports 256K.

7. Errors

Status	Meaning	Fix
401	missing/invalid bearer token	check <code>Authorization: Bearer <token></code>
404 <code>model_not_found</code>	unknown model id	use an id from <code>/v1/models</code> (list is in the error body)
503	model failed to load	check <code>/health</code> ; see <code>journalctl -u vllm-router -f</code> on the box
timeout / connection closed	client gave up during a cold swap	raise client timeout to ≥ 600 s, or use <code>stream: true</code>

8. Quick reference

```
export DGX_API_KEY="<token from models/router-api-key.txt>"
export DGX_BASE="https://spark-e257.tail803c7f.ts.net:8443/v1"

# health (no auth)
curl -s ${DGX_BASE%/v1}/health

# list models
curl -s $DGX_BASE/models -H "Authorization: Bearer $DGX_API_KEY"

# chat
curl -s $DGX_BASE/chat/completions -H "Authorization: Bearer $DGX_API_KEY" \
-H "Content-Type: application/json" \
-d '{"model": "qwen3.6-35b-a3b-nvidia-nvfp4", "messages": [{"role": "user", "content": "hi"}]}'
```

Server-side admin (on the box, not needed by API clients): `vllm-swap <name> | status | list` · `systemctl status vllm-router` · `journalctl -u vllm-router -f`

Hardware: NVIDIA DGX Spark (GB10, 121 GB unified). Config repo: github.com/brishtiteveja/dgx-spark-stack.